```
duck = new Duck
ultraDuckMarathon = new DuckRace [duck]

turnIntoSnake = ->
  duck.walk = null
  duck.slither = ->

turnIntoSnake duck

ultraDuckMarathon.go()
# TypeError: Property walk of object #<AsianDuck> is not a function
```

The walk property of this duck is assigned the value null. It no longer knows how to walk.

You tried to use a type to solve your problem (via the `instanceof` operator) and the result was a type error. Irony.

The `instanceof` operator doesn't promise anything. It doesn't guarantee that an object has a particular interface or works a particular way. It doesn't tell you if something is a duck. All it tells you is which class or constructor the object was created with. In a dynamic language like CoffeeScript, the class or constructor of an object doesn't guarantee anything about what the object actually does right now. The `instanceof` operator is even more brittle because the result will change if you reassign a prototype:

```
class Duck
daffy = new Duck
Duck:: = class Snake

daffy instanceof Duck
# false
```

As a last resort you decide to use the `constructor` property:

```
class Duck
daffy = new Duck
daffy.constructor.name
# Duck
```

That tells you the constructor for the `daffy` object. Again, though, this is a flawed approach in a dynamic language like CoffeeScript. Suppose you create a duck without using a class:

```
duck =
  walk: ->
  quack: ->
```